

Day 4: Does our synthetic-recall harness reproduce the published picture: attention solves MQAR, the delta rule beats additive linear attention, and state size is what the linear mixers are short of?

Version 4.0.2

Research advisor: Claude. Owner: Fan Pu.

September 18, 2026

D1 (spine) · D2 (spine) · backlog item 4 (known-answer reproduction) · sets up H2.1 / H2.3

1 Assignment Overview

Every later comparison of token mixers in this sprint will be judged on two axes: validation loss on web text, and accuracy on a synthetic recall task, because at the model sizes one box can train, loss barely separates mixers while recall does. Today we build the recall axis and, before trusting a single number it produces, check it against results other people have published. The task is multi-query associative recall (MQAR, defined in [Section 2](#)): a sequence lists key-value pairs and later asks for the value of some of the keys. We train tiny two-layer models with three different token mixers on it and ask whether our harness reproduces three published statements: that softmax attention solves the task at a width of 64, that a delta-rule linear mixer (one that corrects its fixed-size memory rather than adding to it; both are defined in [Section 3](#)) does far better than an additive one at the same width, and that both linear mixers improve as their recurrent state grows. The day closes when the first two statements have each been confirmed or contradicted by a measured accuracy with a learning-rate sweep behind it, the third by a width scan at the learning rate that sweep chose, and the recall-versus-state-size plot is in the log.

The setting. Today's GPU budget is about 1.5 GB10-hours by our estimate, 2.9 if every stage takes the maximum its resource line allows (the GB10 is the GPU of the DGX Spark this sprint runs on); that is 0.2% to 0.4% of the sprint's 720 GPU-hours (30 days at 24 hours), and each stage's cost is estimated in [Section 4](#). The goal is to measure, for each of three mixers, the best test accuracy over a grid of four learning rates on one fixed task instance (512 tokens, 64 pairs), and for the two linear mixers the accuracy at three further widths at the learning rate the sweep chose. The constraint that makes the day non-trivial is that the published protocol is large (the original sweep is 448 configurations of 100,000 examples for up to 64 epochs by its README's count, run on eight A100s) and must be cut to fit one box and one day's budget without cutting the parts that make it a reproduction: the task definition, the learning-rate sweep, and the early-stopping rule.

What you will implement

1. `mqar_accuracy`, the accuracy over query positions ([Section 2](#))
2. `naive_linear_attn` and `naive_delta_rule`, the reference recurrences of the two linear mixers ([Section 3](#))
3. `state_elements`, the state-size counter that is the plot's x-axis ([Section 4](#))

What you will run

1. A look at MQAR examples ([Section 2](#)) and the retrieval toy ([Section 3](#))
2. Three smoke runs on the easiest published setting ([Section 6](#))
3. The learning-rate sweep, the width scan, and a second seed ([Section 6](#))

What you will write

1. One pencil-and-paper part, the state sizes (Section 4)
2. Three predictions, before the first run (Section 5)
3. The experiment log and your call on tomorrow's follow-up (Section 7)
4. The finding in your own words (Section 8)

What you can use

- **fla** (flash-linear-attention, v0.5.2): the chunked GPU kernels `chunk_linear_attn` and `chunk_delta_rule`, and `chunk_gated_delta_rule` for the optional ablation (chunked: they process the sequence in blocks of tokens in parallel, and are equivalent to the token-by-token recurrence up to roundoff), called through thin wrappers that fix their options. These are the fast implementations; your references are what they are tested against.
- **torch**: everything else, including `torch.nn.functional.scaled_dot_product_attention` (SDPA, the fused softmax-attention function) for the attention mixer, `torch.optim.AdamW`, and `torch.autocast` (the mixed-precision context).
- **numpy** for the data generator, **matplotlib** for the plot, **tomli-w** (a TOML writer; TOML is the plain-text configuration format each run is described in).
- Written from scratch, by you: the four functions listed above. Each fails silently when wrong: an accuracy that counts unlabeled positions, a recurrence with the scale in the wrong place, a state count off by the number of heads all run to completion and hand you a plausible number. Everything whose failure is loud (data generation, the model, the training loop, the sweep, the plot) is written for you and tagged [AI].

What the code looks like

The zip is a snapshot of `testbed`, the one repository of the sprint. Its layout copies public codebases so that reading the starter teaches you those codebases: the package-beside-tests/ layout and `tests/adapters.py` are the Stanford CS336 starters'; the reference-beside-fast-op pattern (`naive.py` next to `chunk.py`) is `fla`'s; the flat training script is nanoGPT's (Karpathy's minimal GPT trainer); one TOML file per run is torchtitan's (PyTorch's reference pretraining codebase); the sweep-as-a-Python-file and the task generator are Zoology's, the synthetic-recall benchmark code of Arora et al. The README names the counterpart of every file.

1. `testbed/`: the package. `ops/linear_attn/naive.py`, `ops/delta_rule/naive.py`, `evals/mqar_accuracy.py`, and `analysis/state_elements.py` hold one stub each, with the typed signature and docstring from this handout, raising `NotImplementedError`. `tasks/mqar.py` (the generator), `ops/*/chunk.py` (the `fla` wrappers), `layers/attn.py` and `layers/linear_attn.py` (the three mixers), and `models/mqar_lm.py` (the two-layer model) are complete.
2. `tests/`: one test file per function, run as `pytest -k test_<slug>`; `tests/adapters.py` is the glue you fill in so the tests can call your code, and the only file under `tests/` that changes; `tests/test_chunk_ops.py` holds the GPU tests of `fla`'s kernels against your references.
3. `scripts/`: `train_mqar.py` (one run) and `look_at_mqar.py` (examples before training, failures after).
4. `experiments/day4_mqar/`: today's material. One base TOML per mixer at its sweep width (`attn_d64.toml`, `linattn_d128.toml`, `deltanet_d128.toml`, and `gdn_d128.toml` for the gated delta rule, the delta rule with a per-token decay, `gdn` for short, used only by the optional ablation), the smoke presets `smoke_<mixer>_d64.toml`, `sweep.py` (the list of runs per stage: `smoke`, `lr`, `scan`, `scan512`, `seeds`, `gate`, `pivot`; for each run it copies the mixer's base file with the width, learning rate, or training-set size substituted, using `tomli-w`, into `generated/<run name>.toml`, and that generated file is what the run is launched with and what its `config.toml` copies), `run.sh` (runs a stage), `plot.py` (two tables, per run and per mixer and width, and the figure), `log_entry_template.md` (the log skeleton printed in

[Section 7](#)), and `sample_runs/` (four synthetic runs in the exact output format, attention at $d = 64$, the additive rule at $d = 128$, and the delta rule at $d = 128$ at two seeds, for warming up `plot.py`).

5. `examples/retrieval_toy.py`: complete and runnable, walked through in [Section 3](#).
6. `README.md`: setup, tests, each script’s command, expected runtimes on the GB10, and the changelog.
7. `runs/`: created by `train_mqar.py`. Each run writes `runs/<mixer>_d<width>_lr<lr>_s<seed>/` with a copy of its config, an environment file, `metrics.jsonl`, the last test pass’s predictions, the final weights, and `summary.json` when it completes, so that any number in a run directory is reproducible from that directory alone.

Before you start

The following must exist. First, the environment: `torch 2.14.0+cu130`, `fla 0.5.2`, and `TORCH_CUDA_ARCH_LIST="12.1a"` exported, since `fla`’s GPU kernels (written in Triton, a compiler for GPU kernels written in Python) are built for the GB10’s `sm_121` architecture (its CUDA compute-capability name) only when it is set. Second, the starter zip unpacked and `uv sync` run (`uv` is the Python package manager the README uses). Nothing trained on an earlier day is needed: today’s models are trained from scratch on synthetic data, and the only practice carried over is testing a kernel against your own recurrence at the exact shapes you will train with.

Kill and pivot criteria

If, in the smoke stage, the attention model at width 64 does not reach a test accuracy above 0.99 on the easiest published setting (64 tokens, 4 pairs) within 16 epochs, the pipeline is broken, since attention at this width is reported to solve settings eight times harder; the sweep is not launched and today’s result is the diagnosis. If the smoke passes but attention at width 64 fails to exceed 0.99 on the main setting (512 tokens, 64 pairs) at all four learning rates, the question changes from “does the harness reproduce” to “is the reduced protocol the cause”: rerun that one configuration with 100,000 training examples instead of 20,000 at the best of the four learning rates (`run.sh pivot`, one run of about 25 minutes), and the day’s result is whether the published number depends on the training-set size.

Low-Resource Tip: If things go slowly

If the width scan and the second seed cannot be finished within the day’s budget, the reduced result that still closes the day is the sweep alone (Problem (`lr_sweep`)): three mixers at one width each, best over four learning rates, on the main setting, which confirms or contradicts the first two published statements. The width scan and the second seed (Problem (`state_scan`)) are then deferred to whenever the GPU is next free, except for the two $d = 512$ runs, which `run.sh scan512` runs alone in about twenty minutes so that the published delta-rule (DeltaNet) point is still tested; the plot then has two points per linear mixer ($d = 128$ and $d = 512$) instead of four. Nothing changes in `experiments/day4_mqar/`: the `smoke` and `lr` stages, `scan512`, and `plot.py` are the whole reduced day, and the log’s result line carries the sweep’s table.

Who uses this

The first matched comparison of a gated delta-rule mixer (the delta rule of [Section 3](#) with a per-token decay, the mixer of the optional ablation in [Section 6](#)) against the transformer baseline on web text needs a second axis besides loss, and this harness is it: the same models’ mixers are trained on MQAR and their accuracies are read against the state-size axis built today. The study of whether newer variants of the delta rule, which add per-channel decay and separate erase and write gates, close the recall gap at matched state size trains every one of its mixers on this harness, with the learning-rate sweep and early-stopping rule fixed today. The study of how much of the recurrent state is used reads its capacity curve

(accuracy against state size) off the width scan run today. And every recall number reported later in the sprint is trustworthy only to the extent that today’s known-answer check passes.

2 The task: multi-query associative recall

Validation loss on web text averages over every kind of next-token prediction at once, and it is a blunt instrument for comparing mixers: on this sprint’s transformer baselines (34M, 64M, and 127M non-embedding parameters trained on 300M, 600M, and 1.2B tokens of FineWeb-Edu, a public corpus of educational web text; measured earlier in this sprint) the seed alone moves the final validation loss by about 0.003 nats per run (the pooled standard deviation over the six runs, three sizes at two seeds), so with two seeds per arm a gap between mixers of a few thousandths of a nat cannot be told from seed noise; the smallest detectable gap is about 0.008 nats. What separates them is a narrower ability: having seen a key and a value once, produce the value when the key reappears. Multi-query associative recall (MQAR), introduced by Arora et al. [S. Arora et al., 2023, §3], isolates that ability. Here is one example, whole, before any code.

An example is a sequence of L integer tokens from a vocabulary of $V = 8192$ symbols. The first half of the vocabulary, $[1, 4096)$, serves as keys and the second half, $[4096, 8192)$, as values. The first $2n$ positions hold n key–value pairs, key then value, every key distinct and every value distinct. The remaining $L - 2n$ positions are mostly a filler token (0); at n of them, the query positions, a key from the context appears again, and the target at that position is its value. Every other position has no target, marked with the label -100 , which the loss and the accuracy skip. The model reads the sequence left to right (causally) and at each query position must output the value that followed the queried key earlier. Which keys are queried, and how far after the context each query sits, is random: the gap between a key’s first appearance and its query is drawn from a power law that puts most queries soon after the context and a few far away. Today $L = 512$ and $n = 64$, so a model must hold 64 associations at once and answer 64 questions about them, which is the setting in which Yang et al. report their delta-rule result [S. Yang et al., 2024, §4.1].

The generator is `testbed/tasks/mqar.py`, a port of Zoology’s, and it returns two integer tensors of shape (number of examples, L): the inputs and the labels. The training set has 20,000 examples and the test set 3,000, drawn with fixed data seeds (123 and 124) so that every run of the day sees exactly the same examples.

Example (mqar_layout): One example at $V = 12$, $n = 2$, $L = 16$

Keys come from $[1, 6)$, values from $[6, 12)$. With keys (2, 4) and values (8, 7) and the two queries falling at positions 6 and 12:

	Key	Val	Key	Val			Query						Query			
Inputs:	2	8	4	7	0	0	4	0	0	0	0	0	2	0	0	0
Labels:	-100	-100	-100	-100	-100	-100	7	-100	-100	-100	-100	-100	8	-100	-100	-100

Position 6 shows key 4 and its label is 7, the value that followed 4 at position 3; position 12 shows key 2 and its label is 8. The example’s accuracy is the fraction of its two labeled positions at which the model’s most likely next token equals the label: 1 if both, 0.5 if one, 0 if neither. Note that the query key sits *at* the labeled position: the model sees the key and must output the value in the same step.

2.1 Remark: conventions

Width means d , the model dimension, the size of every token's hidden vector; every model today has 2 layers; the linear mixers have 2 heads, so their per-head dimension is $d/2$ for keys and for values alike, and the attention model has 1 head of dimension d . *Accuracy* of a run means the mean over test examples of each example's fraction of correct query positions, the `acc` field of the `eval` records in `metrics.jsonl`; since every example has exactly $n = 64$ queries, this equals the fraction of all query positions answered correctly. *Best accuracy* of a run (`best_acc` in `summary.json`) is the largest test accuracy over its epochs, which is what Zoology reports and what early stopping acts on; *best over learning rates* is the largest `best_acc` among the seed-0 runs of one mixer and width (four in the sweep, one at each scanned width), and is the number every published statement is about. All tensors passed to the linear-attention ops have the layout [B, T, H, D] (batch, time, heads, per-head dimension), which is `fla`'s layout for its chunked kernels. `fla` ships its own naive references, and they are not used here for the reason the ownership split gives: the reference in a correctness test is the piece whose failure is silent, so it is yours. (One of them, the delta-rule reference, also uses [B, H, T, D], so copying it would need a transpose.) *State size* today means the count of numbers a model must hold to keep generating after reading T tokens, summed over its layers ([Section 4](#)); it is a count, not bytes.

Problem (`look_at_mqar`): Look at the data (0.5 points) [you]

Resource requirements: seconds of CPU, no GPU

(a) Run

```
python scripts/look_at_mqar.py --config experiments/day4_mqar/attn_d64.toml
```

It prints the shapes and dtypes, two examples (the first four pairs and the first sixteen positions after the context, input against label), ten random queries with their gaps, and the gap statistics over ten examples.

Deliverable: Three sentences: what the median and maximum gap are and why the median is so much smaller; what a model that ignores the context entirely can score (there are 4096 values); and what, in one sentence, the model has to do at a query position.

The accuracy is the first thing you write, because it is the number everything else today is judged on, and the silent failure is easy: treat an unlabeled position as satisfied (no target, so nothing is wrong there) and divide by all 512 positions, and a model that gets every one of its 64 queries wrong scores $448/512 = 0.875$.

Example (`accuracy`): Two examples, $L = 6$, $V = 5$

Example 0 has labels at positions 2 and 5, values 3 and 4; the model's most likely token there is 3 and 1. Its accuracy is $1/2$. Example 1 has one label, at position 4, value 2, and the model says 2: accuracy 1. At position 0 of example 0 the model happens to output 4, which equals nothing (the label there is -100) and must not count. The result is the tensor $(0.5, 1.0)$, and the run's accuracy would be their mean, 0.75.

Problem (mqar_accuracy): The metric (1 point) [you]

Implement `mqar_accuracy` in `testbed/evals/mqar_accuracy.py`. It is what `train_mqar.py` calls after every epoch, so every accuracy in the sprint comes from it.

```
def mqar_accuracy(logits: torch.Tensor, labels: torch.Tensor) -> torch.Tensor
```

Input

- **logits:** `torch.Tensor` [B, T, V], the model’s output at every position.
- **labels:** `torch.Tensor` [B, T] int64; `-100` everywhere except at query positions, where it is the value token.

Output

- **acc:** `torch.Tensor` [B] fp32: for each example, the number of positions with `labels != -100` at which `logits.argmax(-1) == labels`, divided by the number of positions with `labels != -100`. Unlabeled positions count in neither.

- (a) Implement the function. The test checks Example (accuracy) and an all-right/all-wrong pair.

To test your implementation, implement the adapter `[adapters.run_mqar_accuracy]`. Then, run `pytest -k test_mqar_accuracy` and make sure your implementation passes.

Deliverable: The function and the passing test.

3 Three mixers, one question: what is remembered

The task is fixed; what varies today is the token mixer, the component of each layer that lets position t read from positions before it. Here is the model whole. An input of shape (batch, L) is embedded to (batch, L, d); two identical blocks follow, each of which applies a LayerNorm, the mixer, and a residual addition and nothing else (no MLP, as in the configuration of Arora et al.’s main MQAR figure, the first row of [Table 1](#), whose models have none); dropout of 0.1 is applied to the embeddings and, for attention, to the attention weights; a final LayerNorm and a linear map tied to the embedding matrix produce logits of shape (batch, L, V). The attention model also adds a learned position embedding to the input; the linear mixers get none, since a recurrence knows where it is. The mixer is the only thing that differs between the three models, and the question is what each one remembers about the tokens before t .

Attention. Softmax attention keeps every earlier key and value. At position t it computes a weight over positions $1..t$ from the dot products of its query with every earlier key and returns the weighted sum of the values. To keep generating after T tokens it must hold all T keys and values: $2Td$ numbers per layer, the *KV cache*, growing with T . For MQAR this is the ideal memory: a query key matches its own earlier copy almost exactly and nothing else, so the value that followed it is read out cleanly. Arora et al. report attention solving MQAR at every sequence length they try with a constant width of 64

[\[S. Arora et al., 2023, §4\]](#).

Additive linear attention. Linear attention (Katharopoulos et al. [\[A. Katharopoulos et al., 2020, §3\]](#)) replaces the growing cache by one matrix per head, $S_t \in \mathbb{R}^{d_k \times d_v}$, updated once per token and read once per token:

$$S_t = S_{t-1} + k_t v_t^\top, \quad o_t = S_t^\top q_t. \quad (1)$$

Every pair is *added* into the same matrix. Here is the key piece of math for the section, the identity that everything downstream rests on. Suppose keys $k_1, \dots, k_n$ with values $v_1, \dots, v_n$ have been written and the state is queried with one of them, $q = k_j$:

$$S^\top k_j = \sum_{i=1}^n v_i (k_i^\top k_j) = v_j (k_j^\top k_j) + \sum_{i \neq j} (k_i^\top k_j) v_i. \quad (2)$$

With unit-norm keys the first term is exactly v_j ; the second is the *interference*, every other stored value weighted by how much its key overlaps the query. If the n keys were mutually orthogonal the interference would vanish and the state would be a perfect n -entry table. But a d_k -dimensional space holds at most d_k orthogonal directions, and the keys are learned embeddings of 4095 possible key tokens, any 64 of which may show up together, so they cannot all be orthogonal to each other. Each retrieval is therefore the right value plus a sum of $n - 1$ wrong ones with random-signed coefficients of size roughly $1/\sqrt{d_k}$, and once n approaches d_k that sum is as large as the signal. This is what Arora et al. mean when they write that linear attention alone struggles with associative recall [S. Arora et al., 2024, §1], and it is why the recurrent state’s size, not the parameter count, is the variable a recall comparison must control.

The delta rule. The fix is to write a correction rather than a sum. Before storing v_t , read what the state already predicts for k_t , and write only the difference:

$$S_t = S_{t-1} + \beta_t k_t (v_t - S_{t-1}^\top k_t)^\top, \quad o_t = S_t^\top q_t, \quad (3)$$

with $\beta_t \in [0, 1]$ a per-token gate that says how much of the error to write. This is the delta rule of adaptive filtering (the least-mean-squares update of 1960), brought to linear attention by Schlag et al. [I. Schlag et al., 2021, §4] and made parallel over the sequence by Yang et al. [S. Yang et al., 2024, §3] the model is called DeltaNet. Its guarantee is exact for the newest key: with $\beta_t = 1$, immediately after the write, $S_t^\top k_t = S_{t-1}^\top k_t + (v_t - S_{t-1}^\top k_t)(k_t^\top k_t)$, which is v_t exactly when $|k_t| = 1$. What it does not guarantee is that older entries survive: the write of k_t changes what the state returns for an earlier k_i by $(k_i^\top k_t)$ times the error just written, so overlapping keys still disturb each other, only through their errors rather than through their values. The delta rule uses the space better; it does not make more of it. That is why the day scans width for both linear mixers rather than assuming the delta rule closes the gap on its own. The two rules on a toy, before any of this becomes code: `examples/retrieval_toy.py` writes 12 random unit keys with random values into a 16×16 state with each rule, then queries every key:

```
$ python examples/retrieval_toy.py
d = 16, 12 pairs written; largest |k_i . k_j| between distinct keys = 0.65
relative retrieval error per key (0 = exact), keys in the order written:
  additive: [1.03 0.79 0.94 0.48 0.65 0.94 0.69 0.93 1.21 0.87 0.86 0.95]
  delta:    [1.05 0.9  0.87 0.64 0.56 0.53 0.19 0.25 0.35 0.38 0.15 0. ]
mean error: additive 0.86, delta 0.49; last key: additive 0.95, delta 3.50e-16
```

Twelve keys in sixteen dimensions overlap by up to 0.65, and the additive state returns every value with an error about as large as the value itself. The delta state returns the last key exactly and the recent ones well, and still loses the early ones. The rules of the toy are the recurrences of Equation 1 and Equation 3 with $\beta = 1$.

Problem (retrieval_toy): One knob on the toy (0.25 points) [you]

- (a) Edit the constant on the line `d`, `n = 16`, 12 of `examples/retrieval_toy.py` so that `n` (the number of pairs written) is 4, run again; then 16.

Deliverable: The two mean-error pairs and one sentence on what happens to each rule as n approaches d , and what that predicts for 64 pairs at a per-head dimension of 64.

Example (two_writes): Two writes in two dimensions, then a query

Per head, $d_k = d_v = 2$, $\beta = 1$, and the query is used as is (the ops multiply it by a `scale` first, set to 1 here; the Remark below defines it). Keys $k_1 = (1, 0)$, $k_2 = (0.6, 0.8)$ (unit norm, overlap $k_1^\top k_2 = 0.6$), values $v_1 = (1, 0)$, $v_2 = (0, 1)$. The query at each step is the key just written; a third step queries k_1 again and writes nothing ($v_3 = 0$, $\beta_3 = 0$).

Additive, [Equation 1](#): $S_1 = k_1 v_1^\top = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, $o_1 = S_1^\top k_1 = (1, 0)$. $S_2 = S_1 + k_2 v_2^\top = \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}$, $o_2 = S_2^\top k_2 = (0.6, 1.0)$: the first coordinate should be 0 and is $k_1^\top k_2 = 0.6$, the interference of [Equation 2](#). $o_3 = S_2^\top k_1 = (1, 0.6)$, wrong in the same way.

Delta, [Equation 3](#): $S_1 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ as before (the state predicted 0 for k_1). Before the second write the state predicts $S_1^\top k_2 = (0.6, 0)$ for k_2 ; the error is $v_2 - (0.6, 0) = (-0.6, 1)$, and $S_2 = S_1 + k_2(-0.6, 1) = \begin{pmatrix} 1-0.36 & 0.6 \\ -0.48 & 0.8 \end{pmatrix} = \begin{pmatrix} 0.64 & 0.6 \\ -0.48 & 0.8 \end{pmatrix}$. Now $o_2 = S_2^\top k_2 = (0.384 - 0.384, 0.36 + 0.64) = (0, 1)$: exact. But $o_3 = S_2^\top k_1 = (0.64, 0.6)$: the write of k_2 moved k_1 's entry by $0.6 \times (-0.6, 1)$. Final states: additive $\begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}$, delta $\begin{pmatrix} 0.64 & 0.6 \\ -0.48 & 0.8 \end{pmatrix}$. The tests check these numbers to 10^{-6} .

3.1 Remark: conventions for the ops

Both references take q, k of shape $[B, T, H, K]$, v of shape $[B, T, H, V]$, and the delta rule additionally `beta` of shape $[B, T, H]$. They return the output $[B, T, H, V]$ in the dtype of v and the final state $[B, H, K, V]$ in fp32; the state and all arithmetic are fp32 whatever the input dtype, since bf16 accumulation over 512 steps would make the reference worse than the kernel. The `scale` argument multiplies the query before the read, and nothing else: not the key before the write, and not the key in the delta rule's internal read $S_{t-1}^\top k_t$; `None` means $K^{-1/2}$, which is the kernels' default and what the layer uses, and the tests also pass 1.0 so that the hand example is checked without a $1/\sqrt{2}$. The recurrence starts from a zero state. The layer, not the op, normalizes keys and queries to unit norm and applies the SiLU nonlinearity, and does so identically for both rules; the op sees whatever it is given.

Problem (naive_ops): The two reference recurrences (0.05 GB10 hrs) (1.5 points) [you]

Resource requirements: part (c) only: ≤ 3 minutes of GPU, ≤ 2 GB GPU memory

Write the two references. Each is a loop over T in fp32 with the update and the read of [Equation 1](#) or [Equation 3](#) inside; they are slow (seconds for $T = 512$) and that is fine, since their only job is to be obviously right. They live beside the fast wrappers: `testbed/ops/linear_attn/naive.py` next to `chunk.py`, and `testbed/ops/delta_rule/naive.py` next to its `chunk.py`. The stubs carry the full docstrings.

```
def naive_linear_attn(q, k, v, scale: float | None = None) -> tuple[Tensor, Tensor]
def naive_delta_rule(q, k, v, beta, scale: float | None = None) -> tuple[Tensor, Tensor]
```

Input

- **q, k:** `Tensor [B, T, H, K]`; keys as given, unit-norm when the layer calls, arbitrary in the tests.
- **v:** `Tensor [B, T, H, V]`.
- **beta:** `Tensor [B, T, H]`, delta rule only; the per-token write fraction in $[0, 1]$.
- **scale:** `float | None` Multiplier on q at the read; `None` means $K^{-1/2}$.

Output

- **o:** `Tensor [B, T, H, V]`, dtype of `v`.
- **S:** `Tensor [B, H, K, V]` fp32, the state after the last token.

- (a) `naive_linear_attn`. The test checks Example (two_writes), the default scale, exact retrieval on orthonormal keys, causality (changing inputs after position 7 leaves outputs up to 7 bit-identical), and the dtypes.

To test your implementation, implement the adapter [`adapters.run_naive_linear_attn`]. Then, run `pytest -k test_naive_linear_attn` and make sure your implementation passes.

Deliverable: The function and the passing test.

- (b) `naive_delta_rule`. The test checks Example (two_writes), that $\beta = 0$ writes nothing, that with $\beta = 1$ the key just written is retrieved exactly while the additive rule is off by more than 0.1, that on orthonormal keys the two rules coincide, causality, and the dtypes.

To test your implementation, implement the adapter [`adapters.run_naive_delta_rule`]. Then, run `pytest -k test_naive_delta_rule` and make sure your implementation passes.

Deliverable: The function and the passing test.

- (c) The kernels against your references, at today's shapes. `tests/test_chunk_ops.py` runs `fla's chunk_linear_attn` and `chunk_delta_rule` against your two functions at $(B, T, H, D) \in \{(2, 512, 2, 32), (2, 512, 2, 64), (1, 512, 2, 128), (1, 512, 2, 256)\}$ in bf16 with tolerance 2×10^{-2} (about five units of bf16 roundoff, 2^{-8} , allowing for a sum over 512 terms), checks the four gradients of the delta rule separately in fp32 at 10^{-2} (fp32 inside Triton runs as TF32, a 10-bit-mantissa format, unless told otherwise, so fp32 tolerances are TF32 tolerances), and runs each kernel twenty times on the same input requiring bit-identical outputs. Run `pytest tests/test_chunk_ops.py`. The four per-head dimensions are those of every model trained today, on purpose: a kernel's autotuner (the step that picks block sizes per input shape at first call) picks a different configuration per shape, so a check at another shape checks another kernel.

Deliverable: The pytest summary line, and, if anything fails, the largest absolute error and whether the determinism test passed.

4 Accounting: the known answer, the configurations, and the cost

The mixers are built; before running them we write down exactly what “reproduce” means and what the runs cost, so that a measured number that disagrees with either is noticed. [Table 1](#) lists the three published statements the day tests, each restated in full with its source, and the configuration of ours that tests it. Two of the three are about a different configuration from ours in some detail, and the table says which.

Published statement	Source	Today’s test of it
A two-layer attention model with width 64 reaches near-perfect MQAR accuracy at every sequence length from 64 to 512 (with 4 to 64 pairs; 64 tokens with 4 pairs is the easiest setting), taking the best of four learning rates 10^{-4} to 10^{-2} , 100,000 training examples, up to 64 epochs.	Arora et al. [S. Arora et al., 2023, §4, and Figure 2] the sweep file of their repository, <code>zoology/experiments/paper_configs/iclr24_zoology_figure2/configs.py</code> . The paper’s figure covers 512 tokens, but in the public sweep file the (512, 64) row is commented out, so the file as shipped runs 336 configurations over 64 to 256 tokens; the 512-token attention result is the paper’s, not reproducible from the file as is	Attention, $d = 64$, $L = 512$, $n = 64$, four learning rates; 20,000 examples, up to 32 epochs (the reduction is ours, see the settings below).
DeltaNet with two heads and no short convolution (the causal convolution over the previous few tokens that most linear-attention layers apply to queries, keys, and values before the recurrence) reaches 100% at $L = 512$, $n = 64$ and width 512, and beats Mamba (a selective state-space model, another linear-time mixer) at low widths; gated linear attention (an additive rule with a data-dependent decay) reads at roughly 80% at width 512 on the same figure.	Yang et al. [S. Yang et al., 2024, §4.1, the Figure 4] 80% is read off the plotted curve, not stated in the text	Delta rule at $d \in \{64, 128, 256, 512\}$, two heads, no convolution, same task; the published curve spans the same four widths, but its text states only the $d = 512$ value, so the lower widths are compared by eye against the plotted curve. The published comparators (Mamba, gated linear attention) are not run today; the additive rule without decay stands in for them, so what today tests is the weaker statement that the delta rule beats an additive rule at the same width.
Linear attention alone struggles to solve associative recall, and recall accuracy trades off against recurrent state size across architectures; both linear rules should therefore gain with width.	Arora et al. [S. Arora et al., 2024, §1, a Figures 1–2] qualitative statement, no number	Additive rule at the same four widths as the delta rule; the plot of accuracy against state size.

Table 1: The known answer. “Reads at” marks a value taken from a figure by eye.

Mixer	d	Heads	Per-head $d_k = d_v$	Position embedding	State at $T = 512$, 2 layers
attention	64	1	64	learned, 512×64	$2 \cdot 2 \cdot 512 \cdot 64$
additive, delta	64	2	32	none	$2 \cdot 2 \cdot 32^2$
additive, delta	128	2	64	none	_____
additive, delta	256	2	128	none	_____
additive, delta	512	2	256	none	_____

Table 2: The configurations trained today. Vocabulary 8192, 2 layers, no MLP, no short convolution, embedding tied to the output layer; heads as published (one for attention, two for the linear rules); the blanks are Problem (state_size) (a). The shapes are the [model] tables of the base run files in experiments/day4_mqar/ and of the files sweep.py generates from them for the other widths.

The training settings are the protocol of Arora et al.’s Figure 2 (the first row of Table 1; “Figure 2” below means that protocol) wherever we could afford it, and are given, not derived: deriving a learning-rate grid for three mixers would be a day’s question in itself, and matching the published protocol is the point of a reproduction. Each departure from it is marked.

Learning rates $\{10^{-4}, 4.64 \times 10^{-4}, 2.15 \times 10^{-3}, 10^{-2}\}$. Four points, evenly spaced in $\log_{10}$ from -4 to -2 ; Zoology’s grid, rounded to three significant figures in the generated run files, and the result is the best of the four. Run directories print the rate to two significant figures, so the third point appears as `lr2.2e-03` in a run name.

Optimizer AdamW, weight decay 0.1, cosine decay to 0 over the maximum number of epochs, stepped once per epoch, no warmup. Zoology’s trainer, unchanged. Note that a run that stops early never reaches the end of its cosine.

Early stopping stop after the first epoch whose test accuracy exceeds 0.99. Zoology’s default. It reads the test set, which is unproblematic on synthetic data drawn fresh from a known distribution, and it is why attention runs are cheap: they stop in a few epochs.

Training examples 20,000. A departure: Figure 2 uses 100,000. The protocol Arora et al. used for the same task a year later [S. Arora et al., 2024, §4], as the public sweep file `zoology/experiments/paper_configs/arxiv24_based_figure2/configs.py` has it, uses 20,000 per configuration, and a run at 100,000 examples costs five times as much; the pivot criterion in the overview says what to do if this matters.

Epochs at most 32. A departure: Figure 2 allows 64. Halved for cost; the cosine schedule is over 32, so this is a different schedule, not a truncated one.

Batch 128 examples of $L = 512$ tokens, 65,536 tokens per step; 156 steps per epoch, 4992 steps at 32 epochs. Zoology’s batch size for $L = 512$. The smoke preset uses 256 examples of 64 tokens, 16,384 tokens per step, 78 steps per epoch.

Heads 1 for attention, 2 for the linear rules. Each is its published configuration: Zoology’s attention has one head (so the per-head dimension at $d = 64$ is 64, and the kill criterion rides on exactly the published model), and Yang et al. use two heads for DeltaNet; the additive rule gets two as well so that the two linear rules have the same state size per width.

Precision bf16 autocast, fp32 weights. A departure: Zoology trains in fp32. The sprint’s training runs are bf16 and the harness must match them; the kernel check in Problem (naive_ops) (c) is at bf16 for the same reason.

Dropout 0.1 on the embeddings, 0.1 on attention weights. Zoology’s values; the linear mixers have no attention weights to drop.

Filler the token 0 at every non-query position. Zoology’s Figure 2 setting (`random_non_queries = False`). Random filler is their later default and would make the task slightly harder.

Seeds seed 0 for every run; seed 1 for the three best-learning-rate runs. The seed sets the initialization and the batch order; the data seed is separate and fixed, so all runs see the same examples.

Problem (**state_size**): State size (1 point) [you]

The x-axis of the day's plot. For attention it is the KV cache after T tokens; for a linear mixer it is the state matrices, independent of T .

```
def state_elements(mixer: str, d_model: int, num_heads: int, n_layers: int, seq_len: int) -> int
```

Input

- **mixer:** str "attn", "linattn", "deltanet", or "gdn".
- **d_model, num_heads, n_layers:** int Width, heads, layers; the per-head dimension is $d_model // num_heads$.
- **seq_len:** int Tokens read so far; enters for attention only.

Output

- **count:** int Attention: $n_layers * 2 * seq_len * d_model$. Linear mixers: $n_layers * num_heads * (d_model // num_heads) ** 2$. Numbers held, not bytes.
- (a) Fill the three blanks of [Table 2](#) and compute the attention row's value. Then: a two-layer attention model at $d = 128$ holds how many numbers after 512 tokens, and at which of the four widths does a two-layer linear mixer hold the same count?

Deliverable: Five integers and one sentence naming the width.

- (b) Implement the function in `testbed/analysis/state_elements.py`. The test checks the table's values and that the linear count does not depend on `seq_len`.

To test your implementation, implement the adapter `[adapters.run_state_elements]`. Then, run `pytest -k test_state_elements` and make sure your implementation passes.

Deliverable: The function and the passing test.

Cost. The runtimes below are our estimates, not measurements, and Problem (`lr_sweep`) (b) checks them. One step at $d = 128$ processes 65,536 tokens. The tied output layer is the largest matmul, $(65,536 \times 128)(128 \times 8192)$, which is 6.9×10^{10} multiply-adds or 1.4×10^{11} FLOPs forward and three times that with the backward pass, about 4×10^{11} FLOPs, about 10 ms at an assumed achieved rate of 40 TFLOP/s (dense bf16 peak on this device is about 104 TFLOP/s; small matmuls reach a fraction of it); the logits, $65,536 \times 8192$ bf16 values, are 1.1 GB and are written once and read about twice, 3 GB at the GB10's 273 GB/s memory bandwidth, about 11 ms; the two mixer layers, the embeddings' backward, and the optimizer we round up to the rest. Call it 50 ms per step, 4 minutes for 4992 steps, plus 33 test passes of 3000 examples (one before training, one per epoch) at about a second each: about 5 minutes for a run that goes the distance at $d = 128$, 4 at $d = 64$, 6 at $d = 256$, and 10 at $d = 512$, where the output matmul is four times larger. Runs that stop early cost proportionally less. Peak memory is the logits plus their gradient plus the fp32 slice at the 8192 labeled positions, under 4 GB at $d = 128$; we say ≤ 8 GB. [Table 3](#) sums the stages.

Stage	Runs	Estimated GPU time	Resource line
smoke: (64, 4), $d = 64$, three mixers, one learning rate	3, each 16 epochs of 78 steps at 16,384 tokens	1–2 minutes	≤ 10 min
lr: attention $d = 64$, additive $d = 128$, delta $d = 128$, four learning rates	12; attention stops early	30–45 minutes	≤ 75 min
scan: additive and delta at $d \in \{64, 256, 512\}$, best learning rate	6	30–40 minutes	≤ 60 min
seeds: seed 1 of the three best runs	3	10–15 minutes	≤ 25 min
gate (optional): gated delta rule at $d = 128$, four learning rates	4	15–25 minutes	≤ 45 min
scan512, pivot (contingent): the $d = 512$ pair of scan alone; attention at $d = 64$ with 100,000 examples	2; 1	20 minutes; 25 minutes	≤ 25 min; ≤ 40 min

Table 3: The day’s stages, as `sweep.py` defines them. The compute figure in each Problem’s title is the estimate; the resource line is the allowed maximum. Without the optional stage the estimates sum to about 1.5 hours and the resource lines, with the three minutes of the kernel test, to 2.9.

5 Predictions

Two of the three predictions can be argued from [Table 1](#) and [Equation 2](#); the third is a real unknown. Each answer takes the form the log’s prediction line takes: a value or range, a direction where one applies, and a confidence of high, medium, or low; where a part asks for more than one quantity, give that form for each of them.

Problem (predictions): Three numbers before the first run (1 point) [you]

Log these before proceeding.

- (a) Whether attention reproduces its published result is the pipeline’s known answer; if it fails, nothing else today can be read. The best accuracy over the four learning rates of attention at $d = 64$ on $L = 512$, $n = 64$: the largest `best_acc` among the four `attn_d64_lr*_s0` summaries, a fraction in $[0, 1]$, printed by `plot.py`. Arora et al. report near-perfect accuracy for this width at every length up to 512 under a protocol with five times our examples and twice our epochs ([Table 1](#)).

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry’s prediction line.

- (b) The gap between the delta rule and the additive rule at the same width and state size is the second published statement, and the number the study of newer delta-rule variants named in the overview will build on. Two best-over-learning-rate accuracies at $d = 128$ (per-head dimension 64, 64 pairs), `deltanet` and `linattn`, and their difference, from `plot.py`. Yang et al. plot the delta rule at $d = 128$ on this task but state a value only at $d = 512$, 100%; Equation 2 says the additive rule has 63 interfering terms of relative size about 1/8 each at this per-head dimension; the toy in Section 3 shows the delta rule at $n = 12$, $d = 16$ halving the mean error without removing it.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry’s prediction line.

- (c) How much state the additive rule needs is the third statement, and the shape of the accuracy-against-state-size curve later days will measure on other mixers. The smallest width in {64, 128, 256, 512} at which the additive rule’s best accuracy reaches 0.9, or “none”; and the same for the delta rule. From the per-mixer-and-width table that `plot.py` prints (columns mixer, d , state, best lr, acc s0, other seeds), which covers the `lr` and `scan` stages together. Problem (`state_size`) (a) gives the width at which the linear state matches the $d = 128$ attention model’s KV cache; at $d = 512$ the per-head dimension is 256, four times the number of pairs, and at $d = 64$ it is 32, half of it.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry’s prediction line.

6 Experiments

Two pitfalls come before the script that meets them. First, the number being reported: Zoology’s protocol reports the best test accuracy over epochs and stops at the first epoch above 0.99, so `best_acc` is a maximum over up to 33 noisy evaluations of the test set (the one before training included, at which it is near 0), slightly optimistic for a run that hovers, and `train_mqar.py` records `final_acc` (the last epoch’s) beside it so you can see the difference. We report `best_acc` because the published numbers are `best_acc`. Second, timing: CUDA kernels are launched asynchronously, so a clock read after the backward pass measures launch time; the script synchronizes the device at the end of every epoch and reports tokens per second over training steps only, with the test passes excluded, so its `tok_s` field is comparable with the 1.3×10^6 tokens per second that the estimate in Section 4 implies (65,536 tokens per 50 ms step).

Controls. What varies across the runs: the mixer (three values), the width (four for the linear mixers, one for attention), the learning rate (four values), and, for three runs, the seed. What is held fixed, and why: every value in the settings list of Section 4, so that two runs differ in nothing but the named axes; the training and test examples (data seeds 123 and 124), so that “which examples” is not part of any difference; and the schedule length (32 epochs) whatever the width, so that a wider model is not also a longer-trained one. What the seed controls: the initialization and the order of the training examples. What it does not control: the examples themselves, and the GPU’s summation order, which the determinism test of Problem (`naive_ops`) (c) rules out as a source of variation for the kernels.

Algorithm 1: One MQAR run, as `train_mqar.py` executes it

```
1 Require: a run file (generated/<run name>.toml, written by sweep.py from  
   <mixer>_d<width>.toml), the seed  
2 generate 20,000 training and 3,000 test examples with the data seeds; build the  
3 model with torch.manual_seed(seed); AdamW; cosine over max_epochs  
4 evaluate on the test set; log as epoch -1 (accuracy near 0, loss near  $\ln 8192 =$   
   9.01)  
5 for epoch = 0, ..., 31 do  
6   permute the training examples with the seed's generator  
7   for each batch of 128 do: logits  $\leftarrow$  model(inputs) under bf16 autocast; loss  $\leftarrow$   
   cross-entropy at the labeled positions only; backward; AdamW step  
8   synchronize; evaluate on the test set: loss, and accuracy from your  
   mqar_accuracy; log a train and an eval record  
9   if accuracy > 0.99 then stop; else step the cosine schedule  
10 end for  
11 save the last test pass's predictions and the final weights; write summary.json
```

Problem (train_script): The training script (0.5 points) [AI]

`scripts/train_mqar.py` implements [Algorithm 1](#). Its inputs are `--config` (a run's TOML file), `--seed`, and an optional `--out` directory; every other value, including the learning rate, comes from the file, so the flags of a run are its file and never the command line. It imports your `mqar_accuracy` and nothing else of yours; the state count it records in `summary.json` comes from the layers' own counters, and `plot.py` recomputes the x-axis with your `state_elements` and prints a warning wherever the two disagree. It skips a run whose `summary.json` exists and raises on a non-finite loss.

It writes the run directory with `config.toml` (a verbatim copy of the file it was launched with), `env.json` (the versions of `torch`, `fla`, and `triton`, the GPU kernel compiler `fla` builds on; the GPU name; a hash of the `testbed/` and `scripts/` sources), `metrics.jsonl`, `eval_preds.npz` (the last test pass's most likely token at every position, for the failure inspection), `model.pt` (the final weights, a few megabytes, for evaluation-only follow-ups), and `summary.json` at the end with `best_acc`, `final_acc`, `best_epoch`, `epochs_run`, `early_stopped`, `steps`, `tok_s`, `peak_mem_GB`, `wall_s` (wall-clock seconds of the run), `params`, and `state_elements`; `run.sh` adds `stdout.log`. The smoke stage prefixes its run directories with `smoke_`.

Metrics it logs, in `metrics.jsonl`: a meta record (mixer, width, heads, parameters, state size, steps per epoch); then per epoch a train record (mean training loss over the epoch, learning rate, tokens per second, peak allocated memory, and for the delta rules `beta_mean`, the mean write gate β_t per layer over the last test batch) and an eval record (test loss, test accuracy). Cross-entropy is computed at the 64 labeled positions of each example only; this equals the cross-entropy over all positions with the `-100` labels ignored, and skips materializing fp32 logits at the 448 positions that carry no label. Qualitative samples are not logged; `look_at_mqar.py --run` shows a run's failures instead.

Deliverable: The script, shipped.

Debugging Tip: Three checks, in order, before trusting any accuracy

In a run's `metrics.jsonl`: the epoch `-1 eval` record has `test_loss` within 0.3 of $\ln 8192 = 9.01$ and `acc` below 0.01 (otherwise the labels are wrong or the metric counts unlabeled positions; a model that ignores the context cannot beat $1/4096$); the last `train` record of the run has `train_loss` below 8 (a model that has learned only that answers are value tokens sits at $\ln 4096 = 8.32$; below that it has begun to use the context; a mean stuck near 9 to the end: the learning rate is not applied, or the labels are at the wrong positions); `tok_s` at $d = 128$ is within a factor of two of 1.3×10^6 (our estimate, 65,536 tokens per 50 ms step; far below it, the data are being regenerated or copied per step, or the test pass is inside the timed window).

Problem (smoke): Smoke stage: the easiest published setting (0.03 GB10 hrs) (0.5 points) [you]

Resource requirements: ≤ 10 minutes of GPU for three runs, ≤ 4 GB GPU memory

- (a) Warm up the summary script on the shipped synthetic runs before there are real ones (it calls your `state_elements`, so Problem (`state_size`) comes first):

```
$ python experiments/day4_mqar/plot.py --runs experiments/day4_mqar/sample_runs --out /tmp
```

The four runs are written by a formula and marked `"synthetic": true` in their `summary.json`; their `config.toml` and `env.json` are placeholders. The first table has one row per run with columns `run`, `mixer`, `d`, `lr`, `seed`, `state`, `best_acc`, `final`, `epochs`, `stop`, `tok/s`, `GB`, `min`; the second has one row per mixer and width with columns `mixer`, `d`, `state`, `best lr`, `acc s0`, other seeds. In `/tmp/mqar_recall.png` every run is a faint marker at its state size and best accuracy, and the best seed-0 run per mixer and width (best over learning rates, as the Remark of [Section 2](#) defines it) is a full marker, joined by a line per mixer.

Deliverable: One sentence on which of the four synthetic runs is the faint-only marker and why.

- (b) Run the smoke stage: $L = 64$, $n = 4$, $d = 64$, learning rate 2.2×10^{-3} , 20,000 examples, up to 16 epochs, for all three mixers:

```
nohup experiments/day4_mqar/run.sh smoke > run_smoke.log 2>&1 &
tail -f run_smoke.log
```

Before launching, read the loop of `scripts/train_mqar.py` once, top to bottom. Then apply the first two checks of the Debugging Tip to each of the three runs/

`smoke_<mixer>_d64_lr2.2e-03_s0/metrics.jsonl` files (the third check is about the $d = 128$ configuration and is made in Problem (`lr_sweep`) (b)), and give the evidence that convinced you that the pipeline is correct. Note: it is normal for the accuracy to sit near 0 for the first few epochs and then rise steeply within one or two; the transition, not the slope, is the signature of the task being learned.

Deliverable: One sentence naming the line of the loop at which the labeled positions are selected for the loss; and for each mixer, the epoch at which it stopped or the accuracy it ended at, with the two checked numbers and a pass/fail each. The kill criterion in the overview applies to the attention run only; a linear mixer failing the second check on this easy setting is recorded on the log’s “Noticed but not chased” line and does not stop the day, since the sweep tests it at four learning rates.

Problem (lr_sweep): Learning-rate sweep: the first two published statements (0.6 GB10 hrs) (1 point) [you]

Resource requirements: ≤ 75 minutes of GPU for twelve runs, ≤ 8 GB GPU memory

- (a) Launch the stage; it runs attention at $d = 64$ and the two linear rules at $d = 128$, each at the four learning rates, seed 0, on $L = 512$, $n = 64$:

```
nohup experiments/day4_mqar/run.sh lr > run_lr.log 2>&1 &
```

`python experiments/day4_mqar/sweep.py lr` prints the twelve runs in order; each writes its directory as it goes, so `plot.py` works at any time. If the box dies, re-launch the same command: finished runs are skipped. If a run is much slower than the estimate, look at `tok_s` in its `metrics.jsonl` first and then at whether another process shares the GPU.

Deliverable: Pipeline health, when the twelfth `summary.json` exists: how many runs failed or restarted (from `run_lr.log`), and whether any `train` record has a non-finite loss.

- (b) How fast was it.

Deliverable: The median over the four additive runs at $d = 128$ of the `tok_s` in their `summary.json`, against the 1.3×10^6 estimate, and `wall_s` (from `summary.json`) of one run that went all 32 epochs, the slowest additive run if several did, or the longest run and its `epochs_run` if none did, against the 5-minute estimate. If the measurement is off by more than a factor of two, one sentence on which of the three cost terms in [Section 4](#) was wrong.

- (c) The metric. Run `python experiments/day4_mqar/plot.py` (its defaults are `--runs runs` and `--out experiments/day4_mqar`).

Deliverable: The per-run table (`plot.py` leaves out the `smoke_runs`, so it has twelve rows), and the three best-over-learning-rate rows: attention $d = 64$, additive $d = 128$, delta $d = 128$. Grade predictions (a) and (b) as yes, no, or partially; for (b), partially when the sign of the gap was right and its size was not.

- (d) Read the sweep. For each mixer, is the best learning rate at the edge of the grid? A best value at 10^{-4} or 10^{-2} means the grid did not contain the optimum and the “best over learning rates” is a lower bound.

Deliverable: One sentence per mixer.

- (e) Look at the failures of the best additive run and the best delta run:

```
python scripts/look_at_mqar.py --run runs/<best additive run>
python scripts/look_at_mqar.py --run runs/<best delta run>
```

Each prints ten random failed queries with the key's index in the context (0 = written first), the gap, and whether the wrong prediction is at least a value token, followed by the median key index and gap of the failures against those of all queries.

Deliverable: Two sentences: whether the failures concentrate on early-written keys, on long gaps, or neither, for each rule, and whether that matches the mechanism of [Equation 2](#) and the delta rule's newest-key guarantee.

Problem (state_scan): Width scan and a second seed: the third statement (0.8 GB10 hrs) (1.5 points) [you]

Resource requirements: ≤ 60 minutes of GPU for the six scan runs and ≤ 25 for the three seed runs, ≤ 8 GB GPU memory

- (a) Launch the scan; `sweep.py scan` reads the finished sweep and picks, per linear rule, the learning rate with the highest `best_acc` (the lower rate on a tie), then runs $d \in \{64, 256, 512\}$ at it, seed 0. The learning rate is not re-tuned per width, which Zoology does do; it is a cost decision, and it biases against the widths whose optimum differs from $d = 128$'s.

```
nohup experiments/day4_mqar/run.sh scan > run_scan.log 2>&1 &
```

Then, once it finishes, the second seed of the three best-learning-rate runs:

```
nohup experiments/day4_mqar/run.sh seeds > run_seeds.log 2>&1 &
```

Deliverable: Pipeline health for both stages: runs failed or restarted, non-finite losses.

- (b) The plot. Run `python experiments/day4_mqar/plot.py` and open `experiments/day4_mqar/mqar_recall.png`: best accuracy against state size, one line per mixer through the best seed-0 run per width, every run as a faint marker, the seed-1 runs among them. The spread is shown as the individual runs, not as a mean, since there are two.

Deliverable: The figure and the second table of `plot.py` (mixer, d , state, best lr, acc s0, other seeds) with its seed-1 entries. Grade prediction (c) as yes, no, or partially.

- (c) The seed. For each of the three best runs, the difference in `best_acc` between seed 0 and seed 1.

Deliverable: Three differences, and one sentence: given the variance between the two seeds, how confident are you in the gap between the delta and the additive rule at $d = 128$? Note that the seed band of the sprint's transformer baselines (0.003 nats of validation loss) is a band on a loss, and says nothing about an accuracy; the only accuracy band you have is these three pairs.

- (d) Pass bar for the pipeline, not for the hypothesis: every run completed with finite losses; every epoch -1 test loss within 0.3 of 9.01; the attention smoke run exceeded 0.99. Then the three statements of [Table 1](#), each given one of four verdicts: reproduced, contradicted, not detected, or not tested at the published configuration. The rules: the first is reproduced if attention's best accuracy at $d = 64$ exceeds 0.99, and contradicted otherwise. The second is reproduced if the delta rule's best accuracy at $d = 128$ exceeds the additive rule's by more than 0.2 and by more than twice the largest seed difference of part (c); contradicted if the additive rule's exceeds the delta rule's by that much; and not detected otherwise. The third is reproduced if each linear rule's best accuracy at $d = 512$ exceeds its value at the smallest width run ($d = 64$

on the full day, $d = 128$ on the reduced day of the Low-Resource Tip) by more than 0.2, contradicted if either falls by that much, and not detected otherwise.

Deliverable: Pass or fail on each of the three bars, and three one-line verdicts.

6.1 Ablation 1: the decay gate

The delta rule writes a correction but never forgets; the gated delta rule multiplies the state by a per-token decay $\alpha_t = \exp(g_t) \in (0, 1]$ before each write, which is the mixer most later comparisons in this testbed use (fla’s `chunk_gated_delta_rule`, used as shipped; today’s GPU tests do not cover it, which is one reason the ablation is optional). On a task whose associations must all survive to the end, forgetting can only cost, so this is a one-change check of whether the gate is learned shut.

$$S_t = S_{t-1} + \beta_t k_t (v_t - S_{t-1}^\top k_t)^\top \quad (4)$$

$$S_t = \alpha_t S_{t-1} + \beta_t k_t (v_t - \alpha_t S_{t-1}^\top k_t)^\top \quad (5)$$

Problem (gate): OPTIONAL: Gated delta rule at the same width (0.35 GB10 hrs) (0.5 points) [you]

Resource requirements: ≤ 45 minutes of GPU for four runs, ≤ 8 GB GPU memory

- (a) Run `nohup experiments/day4_mqar/run.sh gate > run_gate.log 2>&1 &`: the gated rule (gdn_d128.toml, the same layer with the decay added, fla’s `chunk_gated_delta_rule`) at $d = 128$, four learning rates, seed 0. Everything else is held fixed at the delta run’s values.

Deliverable: The best-over-learning-rate accuracy beside the delta rule’s at $d = 128$, and two sentences: whether the gate helped, hurt, or did nothing beyond the seed spread of Problem (state_scan) (c), and what that says about whether it was learned shut.

7 What to observe

The numbers. Three best-over-learning-rate accuracies at the sweep stage, then two four-point curves. Attention near 1 is the pipeline check. The delta-additive gap at $d = 128$ is the first result; the width at which each linear curve crosses 0.9 is the second. The seed-1 differences are the band every sentence about a gap has to clear.

The plot. Both linear curves should rise with state size, the delta curve above the additive one at every width if the published picture holds, converging toward 1 at $d = 512$ where the per-head dimension is four times the pair count. A curve that is flat in width says the learning rate, not the state, is the binding constraint at the widths where it is flat; look at whether that width’s best learning rate was at the grid’s edge.

Surprises worth a paragraph. Attention below 0.99 at $d = 64$ with the smoke passed: the pivot applies, and the finding is that the published number needs the published training-set size. The additive rule at or above the delta rule at $d = 128$: this contradicts [Equation 2](#) and the toy at once, and it has two mundane explanations that the logs rule out or not, a rule switch that was not wired (the two runs’ `config.toml` differ only in the mixer line when it was) and a write gate learned near 0 (the delta run’s `beta_mean` in its train records sits well above 0 when it was not), which would make the delta run the additive rule in disguise; with both excluded it is a finding. The delta rule far below 0.9 at $d = 512$: that contradicts the published point, and the learning rate, tuned at $d = 128$ and reused, is the likeliest cause.

A null result is a result. If the delta-additive gap at $d = 128$ is inside the seed spread, the sentence is “we did not detect a difference at this width”, and the width scan says whether it opens at 256 or 512; a gap that only appears at widths where both are near 1 or both near 0 is not a gap. If the width scan is flat for both, the harness is measuring something other than state size at these widths, and that is the finding to log.

Broken pipeline, not a result. A test loss at epoch -1 far from 9.01; an accuracy that is high before the training loss has moved; a `linattn` and `deltanet` pair with identical `metrics.jsonl` (the rule switch is not wired); a determinism test failing in Problem (`naive_ops`) (c).

Problem (`your_call`): Tomorrow's follow-up (0.25 points) [you]

Two candidate follow-ups, each one day: (A) length generalization, evaluation only: reload today's trained models from their `model.pt` files (no retraining) on test sets of $L \in \{1024, 2048\}$ with $n \in \{128, 256\}$ pairs; this is the evaluation half of the protocol of Arora et al. [S. Arora et al., 2024, §4], which trains at 256 tokens with 4 to 64 pairs and tests at up to 1024 tokens with up to 256 pairs, asks whether a state that held 64 pairs holds 256, and costs minutes; (B) the ladder of gates at $d = 128$ on today's task: gated linear attention (additive with decay) and the gated delta rule (if the optional ablation did not run), four learning rates each, about 1 GB10-hour, which turns today's two-rule gap into the four-rule ordering that the study of newer delta-rule variants named in the overview needs.

(a) Choose one and argue for it.

Deliverable: A few sentences, in the log entry.

Problem (`log_entry`): Experiment log (0.5 points) [you]

Fill in the entry below. It is identical to `experiments/day4_mqar/log_entry_template.md`, which carries the fixed lines every day's entry has; today's two extra lines are added by hand. The “What I now believe” paragraph must contain one sentence per published statement of [Table 1](#) saying whether our harness reproduced it, and one sentence on how far the reduced protocol (20,000 examples, 32 epochs, one learning rate per width in the scan) limits that claim. Add two lines of your own after “Spawned question(s):”, named “Noticed but not chased:” and “Your call:”.

```
### Day 4, 2026-09-18
Question: Does our MQAR harness reproduce the published picture at 512 tokens and 64 pairs:
↳ attention solves it at d=64, DeltaNet (delta rule) far above additive linear attention at
↳ d=128, and both linear mixers improving with state size?
Prediction (written BEFORE running): what number do I expect, and how confident?
Setup: MQAR, vocab 8192, 512 tokens, 64 key-value pairs, 20k train / 3k test examples (data
↳ seeds 123 / 124, fixed); 2-layer models, 1 head for attention and 2 for the linear rules, no
↳ MLP, no short conv, tied embeddings; attention at d=64, additive and delta at d=128, four
↳ learning rates {1e-4, 4.6e-4, 2.2e-3, 1e-2}, AdamW wd 0.1, cosine over 32 epochs, batch 128,
```

```

↳ early stop at test acc > 0.99; then d in {64, 256, 512} for both linear mixers at the best
↳ lr; seed 1 of the three best runs; bf16 autocast
Result: (one plot/table, path in repo)
Prediction correct? (yes / no / partially, and what surprised me)
What I now believe:
Spawned question(s):
Hours spent:

```

Deliverable: The filled-in entry, with the three best-over-learning-rate accuracies and the two curves' 0.9 crossings on the result line, the predictions graded item by item on the "Prediction correct?" line, the "Noticed but not chased:" line for anything odd you saw in the logs and did not pursue, and the "Your call:" line from Problem (your_call).

8 Things that are easy to get wrong

- Counting unlabeled positions as correct in the accuracy. A model that gets every query wrong scores $448/512 = 0.875$ that way. The metric counts only the labeled positions, in the numerator and in the denominator.
- Reading "best over learning rates" as a property of the architecture. It is a property of the architecture and the grid; a best value at the grid's edge is a lower bound (Problem (lr_sweep) (d)).
- Carrying the loss band into accuracy. The seed spread of final validation loss measured on this sprint's transformer baselines, about 0.003 nats pooled over three sizes at two seeds, is a spread of losses on web text; it says nothing about how far a synthetic-recall accuracy moves with the seed, which can be tens of points. Today's three seed pairs are the only accuracy band there is.
- Treating the delta rule as capacity. It removes the interference of the newest write, not of older ones (Example (two_writes)); the width scan, not the rule, is what tests capacity.
- Comparing at matched parameters. Attention at $d = 64$ has fewer parameters than the linear mixers at $d = 128$ (about 0.6M against 1.2M, the `params` field of each run's `meta` record) and a state eight times larger (Problem (state_size) (a) gives both counts); the plot's x-axis is state size, and the parameter counts are in every `meta` record for the reader who wants the other matching.
- Believing a kernel because it runs. The GPU test at today's exact shapes is the check; a shape not in it is a kernel not checked.
- Editing a file while a stage is running. Each run is a fresh process that re-imports the package from disk, so a file edited mid-stage changes the later runs and not the earlier ones.

Problem (explain_it_back): Explain it back (0.5 points) [you]

- (a) In four to six sentences, in your own words and without looking at [Section 3](#): what did today's numbers show about the three mixers, why does the additive rule lose associations as the pair count approaches the per-head dimension, what exactly does the delta rule guarantee and not guarantee, and what does "matched state size" mean for a comparison between a linear mixer and attention?

Deliverable: Four to six sentences.

References

- [1] S. Arora *et al.*, “Zoology: Measuring and Improving Recall in Efficient Language Models.” 2023.
- [2] S. Yang *et al.*, “Parallelizing Linear Transformers with the Delta Rule over Sequence Length.” Proc. of NeurIPS, 2024.
- [3] A. Katharopoulos *et al.*, “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.” Proc. of ICML, 2020.
- [4] S. Arora *et al.*, “Simple Linear Attention Language Models Balance the Recall-Throughput Tradeoff.” 2024.
- [5] I. Schlag *et al.*, “Linear Transformers Are Secretly Fast Weight Programmers.” Proc. of ICML, 2021.